숫자 맞추기 게임 개념 정리

1. 게임 개요

컴퓨터가 1부터 100 사이의 숫자를 하나 정하고, 사용자가 5번의 기회 안에 그 숫자를 맞히는 게임입니다. 입력한 숫자가 정답보다 큰지 작은지 힌트를 주고, 기회를 모두 쓰면 정답을 알려 줍니다.

1-1. 구현한 기능

- 게임 시작 때 1~100 사이의 정답 숫자를 무작위로 정함
- [확인] 버튼 또는 Enter 키로 숫자 제출
- 정답보다 크면 "작아요", 작으면 "커요" 힌트 표시
- 남은 기회(5 회)와 지금까지 입력한 숫자 표시
- 빈 칸, 실수, 1~100 밖의 숫자는 기회를 쓰지 않고 안내만 표시
- 정답을 맞히거나 기회를 모두 쓰면 입력 막기 + [다시 시작] 버튼 표시

1-2. 파일 구성

파일	역할
index.html	기회 표시, 숫자 입력창과 버튼, 힌트 메시지, 입력 기록, 다시 시작 버튼의 뼈대
css/style.css	카드 디자인, 입력이 막힌 상태(:disabled), 결과 색(.win / .lose), 버튼 표시(.show)
js/script.js	게임 상태 변수, 게임 시작·종료·확인 함수, 이벤트 연결

2. 핵심 아이디어: 게임의 "상태"를 변수에 담는다

이 게임은 시간이 지나면서 바뀌는 값 세 가지로 움직입니다. 이런 값을 "상태"라고 부르고, 화면은 이 상태를 보여 주는 결과일 뿐입니다.

변수	의미	언제 바뀌는가
`answer`	정답 숫자	게임을 시작할 때 한 번 정해짐 (다시 시작하면 새로 정함)
`chances`	남은 기회	올바른 숫자를 제출할 때마다 1씩 줄어듦
`tried`	입력한 숫자 기록	제출할 때마다 뒤에 이어 붙음

순서	무엇을 하는가	코드
① 이벤트	사용자가 숫자를 제출함	버튼 click 또는 Enter keydown
② 상태 변경	변수 값을 바꾸고 결과를 판단함	chances = chances - 1, if / else if
③ 화면 반영	바뀐 값을 화면에 보여 줌	textContent, classList, disabled

3. HTML — 새로 나오는 태그와 속성

앞 수업에서 배운 태그(header, section, nav, form, label, input, button, p, strong 등)는 제외하고 정리합니다.

3-1. <main> — 페이지의 핵심 내용 영역

header, section, footer처럼 문서의 의미를 알려 주는 시맨틱 태그입니다. 페이지에서 가장 중요한 내용(이 예제에서는 게임 카드 전체)을 감쌉니다. 한 페이지에 한 번만 사용합니다.

```
<body>
    <main class="game">
        <h1>Number Guess</h1>
        ...
    </main>
</body>
```

- 모양은 div 와 같지만, 브라우저와 검색 엔진에 "여기가 본문"이라고 알려 주는 역할을 합니다.

3-2. <input type="number">와 min, max 속성

```
<input class="guess-input" type="number" min="1" max="100" placeholder="1 ~ 100">
```

속성	설명
type="number"	숫자만 입력할 수 있는 입력창입니다. 오른쪽에 위·아래 화살표가 생기고, 스마트폰에서는 숫자 키패드가 나옵니다.
min="1" max="100"	입력할 수 있는 최솟값과 최댓값을 정합니다. 화살표를 눌러 올리거나 내릴 때는 이 범위를 지킵니다.

주의: min, max는 화살표 버튼에만 적용되고, 사용자가 직접 타이핑한 값은 막지 못합니다. 또한 type="number"여도 JavaScript에서 읽는 값(input.value)은 숫자가 아니라 글자입니다. 그래서 JavaScript에서도 범위 검사를 다시 해야 합니다. (5-4 참고)

3-3. disabled 속성

```
<!-- HTML에 직접 쓰면 처음부터 막힌 상태 -->
<button disabled>확인</button>


// JavaScript에서 켜고 끌 수 있다.
guessBtn.disabled = true;     // 막기
guessBtn.disabled = false;    // 풀기
```

disabled가 붙은 입력창과 버튼은 클릭하거나 입력할 수 없습니다. 이 예제에서는 HTML에 쓰지 않고, 게임이 끝났을 때 JavaScript가 붙입니다. 값은 true / false로 바꿉니다.

4. CSS — 새로 사용되는 속성

앞 수업에서 배운 속성(변수, 선택자, 박스 모델, flex, 배경, hover, transition, 미디어 쿼리 등)은 제외하고 정리합니다.

4-1. 전체 선택자(*)와 box-sizing

```
* { box-sizing: border-box; }
```

- *** (전체 선택자):** 페이지의 모든 요소를 한 번에 선택합니다.
- **box-sizing: border-box:** width 에 padding 과 border 를 포함해서 계산합니다.

기본값(content-box)에서는 width: 100%에 padding을 더하면 부모 상자를 넘쳐 나갑니다. border-box로 바꾸면 padding을 넣어도 지정한 너비 안에 들어와 레이아웃 계산이 쉬워집니다.

값	width: 200px; padding: 20px 일 때 실제 가로 크기
content-box (기본)	200 + 20 + 20 = 240px (padding이 바깥으로 늘어남)
border-box	200px (padding이 안쪽에 포함됨)

4-2. html 의 font-size 로 전체 글자 크기 조절

```
html { font-size: 125%; }    /* 브라우저 기본 16px × 125% = 20px */
```

rem 단위는 html 요소의 글자 크기를 기준으로 합니다. 따라서 html의 font-size 한 줄만 바꾸면 rem으로 지정한 모든 글자와 여백이 함께 커지거나 작아집니다. 글자가 작다는 의견이 있을 때 가장 먼저 조절하기 좋은 곳입니다.

4-3. min-height 와 각도가 있는 그라데이션

```
body {
    min-height: 100vh;
    background: linear-gradient(160deg, #efeaff, #fff1e6);
}
```

속성	설명
min-height: 100vh	높이가 최소 화면 높이만큼은 되도록 합니다. height와 달리 내용이 많으면 그만큼 늘어납니다. 배경이 화면 전체를 채우되 내용이 넘쳐도 잘리지

속성	설명
	않습니다.
`linear-gradient(160deg, 색1, 색2)`	앞에 각도를 쓰면 그라데이션 방향이 정해집니다. 180deg는 위→아래, 90deg는 왼쪽→오른쪽이며, 160deg는 거의 위→아래로 약간 비스듬합니다.

4-4. box-shadow — 그림자

```
.game {
    box-shadow: 0 14px 40px rgba(34, 48, 90, .18);
}
```

순서	값의 의미
1번째	X 방향 이동 (0 = 이동 없음)
2번째	Y 방향 이동 (14px = 아래쪽으로)
3번째	흐림 정도 (클수록 부드럽게 퍼짐)
4번째	그림자 색. rgba의 마지막 숫자가 투명도이며 작을수록 연합니다.

카드가 배경에서 떠 있는 것처럼 보이게 합니다. 그림자는 진하게 쓰기보다 투명도를 낮춰 은은하게 쓰는 것이 깔끔합니다.

4-5. flex 자식 요소의 크기: flex, min-width

```
.guess-row   { display: flex; gap: .5rem; }
.guess-input { flex: 1; min-width: 0; }     /* 입력창이 남는 공간을 모두 차지 */
.guess-btn   { padding: 0 1.6rem; }         /* 버튼은 자기 크기만큼만 */
```

속성	설명
`flex: 1`	남는 공간을 모두 차지합니다. 입력창에 쓰면 버튼은 자기 크기만 갖고, 나머지는 입력창이 채웁니다.
`min-width: 0`	flex 자식(특히 input)은 기본적으로 내용보다 작아지지 못해 화면이 좁을 때 부모를 넘칠 수 있습니다. 0으로 지정하면 줄어들 수 있게 됩니다.

4-6. :focus — 입력창을 선택했을 때의 모양

```
.guess-input:focus {
    outline: none;
    border-color: var(--purple);
}
```

:hover가 "마우스를 올렸을 때"라면, :focus는 "클릭하거나 Tab 키로 선택되어 커서가 들어와 있을 때"입니다. 브라우저는 포커스된 입력창에 기본 윤곽선(outline)을 그려 주는데, outline: none으로 이를 지우고 테두리 색을 바꾸어 직접 디자인합니다. 포커스 표시를 완전히 없애면 사용자가 어디에 입력 중인지 알 수 없으므로 반드시 다른 표시(테두리 색 등)를 대신 넣어야 합니다.

4-7. :disabled — 막힌 상태의 모양

```
.guess-input:disabled,
.guess-btn:disabled {
    opacity: .4;
    cursor: not-allowed;
}


.guess-btn:disabled:hover {      /* 막힌 버튼은 hover 색이 바뀌지 않게 */
    background: var(--purple);
}
```

코드	설명
`:disabled`	disabled 속성이 붙은 요소만 선택하는 가상 클래스입니다. :hover, :focus와 같은 종류입니다.
`opacity: .4`	요소 전체를 투명하게 만들어 흐리게 보이게 합니다. (0 = 완전 투명, 1 = 원래대로)
`cursor: not-allowed`	마우스를 올리면 금지 모양(🚫) 커서가 나옵니다.
`:disabled:hover`	가상 클래스를 이어서 쓰면 "막힌 상태에서 마우스를 올렸을 때"만 선택합니다.

4-8. 상태 클래스 패턴: .win, .lose, .show

CSS에서는 "이겼을 때 모양", "보일 때 모양"을 미리 클래스로 만들어 두고, JavaScript가 classList.add / remove로 그 클래스를 붙였다 뗍니다. 탭 메뉴의 .active와 같은 방식입니다.

```
.message.win  { color: var(--green); }   /* message 와 win 을 모두 가진 요소 */
.message.lose { color: var(--red); }


.restart-btn      { display: none; }      /* 기본은 숨김 */
.restart-btn.show { display: block; }     /* .show 가 붙으면 표시 */
```

- 띄어쓰기 없이 이어 쓴 `.message.win` 은 "message 와 win 클래스를 동시에 가진 요소"라는 뜻입니다.
- HTML 은 바꾸지 않고, 클래스를 붙였다 떼는 것만으로 색과 표시 여부를 조절할 수 있습니다.

5. JavaScript — 새로 배우는 개념

지금까지 배운 querySelector, addEventListener, classList, textContent 위에 아래 개념들이 새로 더해집니다.

5-1. 변수: let 과 const

변수는 값을 담아 두는 이름 붙은 상자입니다. 값이 바뀌어야 하는지에 따라 키워드를 구분합니다.

키워드	의미	이 예제에서
const	한 번 정하면 다시 대입할 수 없는 변수	guessInput, message 등 요소 선택 결과, MAX_CHANCES (5)
let	나중에 값을 다시 대입할 수 있는 변수	answer, chances, tried (게임 중 계속 바뀜)

```
const MAX_CHANCES = 5;      // 바뀌지 않는 값은 const (이름을 대문자로 쓰는 것이 관례)


let chances = MAX_CHANCES; // 바뀌는 값은 let
chances = chances - 1;     // 대입 연산자 =  : 오른쪽 계산 결과를 왼쪽 변수에 넣는다.
chances = chances - 1;     // 다시 대입 가능 → 5 → 4 → 3
```

- 수학의 등호가 아니라 "오른쪽 값을 왼쪽 변수에 넣어라"라는 뜻입니다. `chances = chances - 1` 은 "현재 chances 에서 1 을 뺀 값을 다시 chances 에 넣어라"입니다.
- const 변수에 다시 대입하면 오류가 납니다. 그래서 바뀌는 값에는 반드시 let 을 씁니다.
- 값이 없는 상태로 선언만 할 수도 있습니다. `let answer;` (나중에 startGame 에서 값을 넣음)

글자 이어 붙이기: 글자에 + 를 쓰면 더하기가 아니라 이어 붙이기가 됩니다.

```
tried = tried + guess + '  ';     // '' + 70 + '  '  → '70  '
                                     // '70  ' + 20 + '  ' → '70  20  '


'입력한 숫자 : ' + tried             // 글자 + 글자 = 이어 붙인 글자
```

주의: 숫자끼리의 +는 더하기이고, 한쪽이 글자이면 이어 붙이기입니다. 화면에 "정답은 50 였습니다"를 표시할 때처럼 글자와 숫자를 섞어 쓸 때 활용합니다.

5-2. 함수 선언과 호출

함수는 여러 줄의 코드에 이름을 붙여 필요할 때마다 부를 수 있게 한 것입니다. 이 예제의 핵심은 "같은 일을 여러 곳에서 해야 할 때 함수로 만들어 재사용한다"는 점입니다.

```
function startGame() {      // 선언: 이름을 붙여 코드를 묶어 둔다. (아직 실행 안 됨)
  answer = ...;
  chances = MAX_CHANCES;
  ...
}


startGame();                  // 호출: 이때 실제로 실행된다.
```

함수	하는 일	호출되는 곳
`startGame()`	정답을 새로 정하고 화면을 처음 상태로 되돌림	페이지를 열 때, [다시 시작] 클릭
`checkGuess()`	입력한 숫자를 검사하고 결과를 판단	[확인] 클릭, Enter 키
`endGame()`	입력을 막고 다시 시작 버튼을 표시	정답을 맞혔을 때, 기회를 모두 썼을 때

- **선언만 하고 호출하지 않으면 아무 일도 일어나지 않습니다.** 이 예제 맨 아래의 `startGame();`가 페이지를 열자마자 게임을 시작시킵니다.
- **이벤트에 함수를 연결할 때는 괄호를 쓰지 않습니다.** `guessBtn.addEventListener('click', checkGuess)`처럼 이름만 쓰면 "클릭할 때 이 함수를 실행해라"라는 뜻이고, `checkGuess()`처럼 괄호를 쓰면 지금 바로 실행해 버립니다.

5-3. 난수 만들기: Math.random, Math.floor

정답 숫자를 컴퓨터가 무작위로 정하기 위해 Math 객체의 기능을 사용합니다.

코드	설명
`Math.random()`	0 이상 1 미만의 무작위 소수를 돌려줍니다. (예: 0.7342...) 1은 나오지 않습니다.
`Math.floor(수)`	소수점 아래를 버립니다. (예: 73.42 → 73)

```
answer = Math.floor(Math.random() * 100) + 1;
```

단계	결과 범위
`Math.random()`	0 ≤ 수 < 1
`Math.random() * 100`	0 ≤ 수 < 100 (예: 73.42)
`Math.floor(... )`	0, 1, 2, ... 99 (소수점 버림)
`... + 1`	1, 2, 3, ... 100 ← 우리가 원하는 범위

응용 공식: a부터 b까지의 정수는 `Math.floor(Math.random() * (b - a + 1)) + a`입니다. 주사위(1~6)는 `Math.floor(Math.random() * 6) + 1`입니다.

5-4. 입력값 다루기: Number(), Number.isInteger()

input.value는 항상 글자(문자열)입니다. type="number" 입력창에서 읽어도 마찬가지입니다. 숫자로 계산하거나 비교하려면 Number()로 바꿔야 합니다.

```
const guess = Number(guessInput.value);    // '50' → 50


// 글자끼리의 비교는 사전 순서이므로 우리가 기대한 결과와 다르다.
'9' > '10'      // true   ('9'가 '1'보다 뒤에 오기 때문)
9 > 10          // false  (숫자로 비교해야 맞다.)
```

코드	설명
`Number('50')`	글자 50을 숫자 50으로 바꿉니다. 빈 글자는 0, 숫자가 아닌 글자는 NaN(숫자 아님)이 됩니다.
`Number.isInteger(수)`	정수이면 true, 소수(3.5)이거나 NaN이면 false입니다.

빈 칸 검사를 따로 하는 이유: Number('')은 0이 되어 1~100 범위 검사에서 걸리기는 하지만, 빈 칸일 때 "숫자를 입력하세요"라는 안내가 정확히 나오도록 value === '' 검사를 함께 둡니다.

5-5. 조건문: if / else if / else

조건이 참인지 거짓인지에 따라 실행할 코드를 고릅니다. 위에서부터 차례로 검사하다가, 처음으로 참인 한 곳만 실행하고 나머지는 건너뜁니다.

```
if (guess === answer) {          // 1. 정답?
  ...정답 처리...
} else if (chances === 0) {        // 2. 틀렸고 기회도 없음?
  ...실패 처리...
} else if (guess > answer) {       // 3. 입력이 더 큼?
  ...작아요 힌트...
} else {                           // 4. 위가 모두 아니면 (입력이 더 작음)
  ...커요 힌트...
}
```

연산자	의미	예
===	값과 종류가 모두 같다	guess === answer
!==	같지 않다	text !== ''
> <	크다 / 작다	guess > answer, guess < 1
>= <=	크거나 같다 / 작거나 같다	chances >= 1

- **= 와 === 의 차이:** = 는 값을 넣는 대입, === 는 같은지 묻는 비교입니다. if 안에서 = 를 쓰면 의도와 다르게 동작합니다.
- **검사 순서가 중요합니다.** 정답 검사를 맨 뒤에 두면 정답을 맞혔는데도 "커요/작아요"가 먼저 나올 수 있습니다. 또 "기회 소진" 검사를 "큰지/작은지"보다 앞에 두어야 마지막 시도에서 틀렸을 때 실패 메시지가 나옵니다.

5-6. 논리 연산자: ||, &&, !

여러 조건을 하나로 묶을 때 사용합니다. 입력값 검사에서 "이런 경우 중 하나라도 해당하면 잘못된 입력"을 표현합니다.

연산자	이름	의미
||	또는 (OR)	둘 중 하나라도 참이면 참
&&	그리고 (AND)	둘 다 참이어야 참
!	아님 (NOT)	참과 거짓을 뒤집음

```
if (guessInput.value === '' || !Number.isInteger(guess) || guess < 1 || guess > 100)
{
  // 빈 칸이거나, 정수가 아니거나, 1보다 작거나, 100보다 크면 → 잘못된 입력
}
```

네 조건 중 하나라도 참이면 전체가 참이 되어 안내 메시지를 보여 줍니다. 모두 거짓일 때만 올바른 입력으로 보고 아래 코드로 넘어갑니다.

5-7. return — 함수를 중간에 끝내기

```
function checkGuess() {
  if (잘못된 입력) {
    message.textContent = '1부터 100 사이의 정수를 입력하세요.';
    return;                  // 여기서 함수를 끝낸다. 아래 코드는 실행되지 않는다.
  }
  chances = chances - 1;     // 올바른 입력일 때만 도달한다.
  ...
}
```

잘못된 입력일 때는 기회를 깎지 않고 안내만 하고 싶습니다. return을 만나면 함수가 즉시 끝나므로, 그 아래의 "기회 차감" 코드가 실행되지 않습니다. 검사에서 걸러 내는 코드에서 자주 쓰는 패턴입니다.

5-8. 요소의 상태 바꾸기: disabled, value, focus()

코드	설명
`guessInput.disabled = true`	입력창을 막습니다. false로 바꾸면 풀립니다. 버튼도 같은 방식입니다.
`guessInput.value = ''`	입력창에 쓰인 글자를 지웁니다. (읽을 때는 .value, 바꿀 때도 .value)
`guessInput.focus()`	입력창에 커서를 둡니다. 마우스로 다시 클릭하지 않아도 바로 다음 숫자를 입력할 수 있습니다.
`message.classList.remove('win', 'lose')`	classList.remove에는 클래스를 여러 개 한꺼번에 쓸 수 있습니다.

textContent에 숫자를 넣어도 됩니다. chanceCount.textContent = chances;처럼 숫자를 넣으면 화면에는 글자로 표시됩니다.

5-9. 키보드 이벤트: keydown, event.key

```
guessInput.addEventListener('keydown', (event) => {
  if (event.key === 'Enter') {
    checkGuess();
  }
});
```

코드	설명
`keydown`	키보드의 키를 누르는 순간 발생하는 이벤트입니다. (키를 뗄 때는 keyup)
`event.key`	눌린 키의 이름입니다. Enter 키는 'Enter', 알파벳 a는 'a', 숫자 5는 '5'입니다.

모든 키에 반응하면 안 되므로, if로 "Enter일 때만" 실행합니다. checkGuess 함수를 따로 만들어 두었기 때문에 버튼 클릭과 Enter 키 두 곳에 같은 함수를 연결할 수 있습니다.

참고: form 태그를 사용하면 submit 이벤트 하나로 Enter 키와 버튼 클릭을 한 번에 처리할 수 있습니다. 이 방법은 다음 시간의 To-do List에서 배웁니다.

5-10. 이벤트 종류 정리

이벤트	발생 시점	이 예제에서
click	요소를 클릭할 때	[확인], [다시 시작] 버튼
keydown	키를 누를 때	입력창에서 Enter 키

6. script.js 흐름 따라가기

번호	블록	설명
1	요소 선택	querySelector로 입력창, 버튼, 메시지, 기회 표시 등을 변수에 저장합니다.
2	변수 선언	const MAX_CHANCES, let answer / chances / tried
3	startGame	정답 정하기 → 기회·기록 초기화 → 화면과 입력창 처음 상태로
4	endGame	입력창과 버튼 막기 → 다시 시작 버튼 표시
5	checkGuess	입력 변환·검사 → 기회 차감 → 정답/실패/힌트 판단 → 입력창 비우기
6	이벤트 연결	확인 버튼 click, Enter keydown → checkGuess / 다시 시작 click → startGame
7	startGame() 호출	페이지를 열자마자 첫 게임을 시작합니다.

checkGuess 안의 순서: ① 숫자로 변환 → ② 잘못된 입력이면 안내하고 return → ③ 기회 1 차감, 기록 추가 → ④ if / else if로 결과 판단 → ⑤ 입력창 비우고 커서 이동

7. 자주 하는 실수

실수	결과와 해결
input.value를 Number()로 바꾸지 않고 비교	'9' > '10'이 true가 되는 것처럼 글자 순서로 비교되어 힌트가 틀립니다. Number()로 바꿉니다.
if 안에서 = 하나만 사용	비교가 아니라 대입이 되어 항상 참이 되거나 오류가 납니다. 비교는 ===를 씁니다.
정답 검사를 else if의 맨 뒤에 둠	정답을 맞혔는데도 힌트 메시지가 나옵니다. 가장 먼저 검사해야 하는 조건부터 위에 씁니다.
const chances = 5; 로 선언하고 chances = chances - 1	오류가 발생합니다. 값이 바뀌는 변수는 let으로 선언합니다.
addEventListener('click', checkGuess())	괄호 때문에 페이지를 열 때 바로 실행되고 클릭에는 반응하지 않습니다. 함수 이름만 씁니다.
함수만 선언하고 startGame()을 호출하지 않음	게임이 시작되지 않아 정답이 undefined 상태입니다. 맨 아래에서 한 번 호출합니다.
잘못된 입력에서 return을 빠뜨림	안내 메시지를 보여 주고도 기회가 차감됩니다. 안내 후 return으로 함수를 끝냅니다.
Math.random() * 100만 사용	소수가 나오고 1부터 100이 아닙니다. Math.floor(...) + 1을 함께 씁니다.